

GeCode CSP Pipeline - Manual Completo

Sistema de Procesamiento de Problemas de Constraint Programming

Proyecto GeCode CSP Pipeline

2026

Contents

Introduccion	7
?Que es GeCode CSP Pipeline?	7
Proposito del Proyecto	7
Caracteristicas Principales	7
? Validacion y Analisis	7
? Propagacion de Restricciones	7
? Resolucion CSP	7
? Persistencia y Utilidades	7
? Compilacion Flexible	7
Tipos de Datos Soportados	8
Operadores Soportados	8
Aritmeticos	8
Logicos	8
Relacionales	8
Conjuntos	8
Funciones Matematicas	8
Compilacion e Instalacion	10
Requisitos del Sistema	10
Para Herramientas del Pipeline	10
Para Integracion con Gecode	10
Instalacion de Dependencias	10
Ubuntu/Debian	10
Fedora/RHEL	10
Instalacion de Gecode Estatico	10
Compilacion del Proyecto	11
Compilacion Completa	11
Compilacion Selectiva	11
Compilacion Monolitica	11
Verificacion de la Instalacion	12
Inicio Rapido	13
Ejemplo Basico	13

1. Crear un archivo JSON de entrada	13
2. Ejecutar el pipeline	13
3. Revisar resultados	13
Pipeline con Persistencia	13
Uso Individual de Componentes	14
Explorar Ejemplos	14
Proximos Pasos	15
Vision General de la Arquitectura	16
Filosofia de Diseno	16
Principios de Diseno	16
Arquitectura de Alto Nivel	16
Componentes del Sistema	18
1. SyntaxChecker	18
2. JsonToGraph	18
3. FunctionChecker	18
4. FwdConsistency	18
5. BwdConsistency	19
6. TestGecodeBridge	19
Herramientas Auxiliares	19
CSPEval	19
ForwardChain	19
JsonSink / JsonSource	19
Flujo de Datos	20
Formato JSON de Entrada	20
Transformaciones por Etapa	20
Etapa 1: SyntaxChecker -> JSON Validado	20
Etapa 2: JsonToGraph -> Grafo con AST	21
Etapa 3: FunctionChecker -> Grafo Verificado	21
Etapa 4-5: Propagacion -> Dominios Reducidos	21
Etapa 6: TestGecodeBridge -> Soluciones	22
Arquitectura de Persistencia	23
SQLite como Almacenamiento Intermedio	23
Esquema de Base de Datos	23
Flujo con Persistencia	23
Ventajas de la Persistencia	23
Uso	23
Integracion de Lenguajes	24
Pascal + C/C++ + Gecode	24
1. Pascal (Free Pascal)	24
2. C (Biblioteca MiniMath)	24
3. C++ (Gecode Bridge)	24
Diagrama de Integracion	24
Sistema de Compilacion	26

Makefile Modular	26
Compilacion Monolitica	26
Modos de Linkeo	26
Diagrama de Compilacion	27
Componentes del Pipeline	28
Flujo General	28
1. SyntaxChecker	29
Proposito	29
Entrada/Salida	29
Reglas de Validacion	29
Codigos de Salida	29
Uso	29
Ejemplo	30
Tecnologias	30
Posicion en el Pipeline	30
2. JsonToGraph	31
Proposito	31
Entrada/Salida	31
Algoritmo Principal	31
Tipos de Nodo AST	31
Nodos de Valor	31
Nodos Relacionales	31
Nodos Logicos	31
Nodos Aritmeticos	32
Nodos Especiales	32
Uso	32
Ejemplo de Transformacion	32
Tecnologias	33
Posicion en el Pipeline	33
3. FunctionChecker	34
Proposito	34
Entrada/Salida	34
Algoritmo de Busqueda	34
Directorios de Busqueda	34
Codigos de Salida	34
Uso	35
Notas	35
Tecnologias	35
Posicion en el Pipeline	35
4. FwdConsistency	36
Proposito	36
Entrada/Salida	36
Algoritmo AC-3	36
Ejemplo de Paso	36

Uso	37
Tecnologias	37
Posicion en el Pipeline	37
5. BwdConsistency	38
Proposito	38
Entrada/Salida	38
Algoritmo HC4-Revise	38
Diferencia con FwdConsistency	38
Ejemplo	38
Uso	39
Tecnologias	39
Posicion en el Pipeline	39
6. CSPEval	40
Proposito	40
Entrada/Salida	40
Tipos de Variable Soportados	40
Algoritmo de Propagacion	40
Ejemplo de Salida	40
Diferencia con FwdConsistency/ForwardChain	41
Uso	41
Tecnologias	41
Posicion en el Pipeline	41
7. ForwardChain	42
Proposito	42
Entrada/Salida	42
Algoritmo de Encadenamiento	42
Estados de Retorno	42
Ejemplo de Steps	42
Diferencia con FwdConsistency	43
Uso	43
Tecnologias	43
Posicion en el Pipeline	43
8. JsonSink	44
Proposito	44
Entrada/Salida	44
Esquema SQLite	44
Funcionalidad	44
Uso	44
Notas	45
Tecnologias	45
Posicion en el Pipeline	45
9. JsonSource	46
Proposito	46
Entrada/Salida	46

Modos de Recuperacion	46
Casos de Uso Tipicos	46
1. Guardar y Reutilizar Grafo	46
2. Resolver CSP desde Grafo Guardado	46
3. Recuperar Resultado Anterior	46
Notas	46
Tecnologias	47
Posicion en el Pipeline	47
Resumen de Componentes	48
Tabla Comparativa	48
Pipelines Tipicos	48
Pipeline Completo con Propagacion	48
Pipeline con Persistencia	48
Pipeline de Evaluacion Rapida	48
Introduccion a la Integracion	50
Vision General	50
Arquitectura del Puente Pascal/C++	52
Componentes del Puente	52
1. UGecodeBridge.pas (Lado Pascal)	52
2. gecode_bridge.cpp (Lado C++)	52
Flujo de Datos	52
Estructuras de Datos Compartidas	53
TCSPVar / Variable	53
TCSPConstraint / Constraint	53
TCSPSolution / Solution	54
API Exportada desde C++	55
Funciones Principales	55
csp_create	55
csp_add_constraint	55
csp_solve_first	55
csp_solve_all	56
csp_count_solutions	56
csp_count_with_constraint	56
csp_debug_domains	56
csp_free	57
Tipos de Restricciones Soportadas	58
Restricciones Basicas	58
Restricciones de Dominio	58
Restricciones Lineales	58
Restricciones de Valor Absoluto	59
Restricciones de Distancia	59
Restriccion Global	59

Compilacion Monolitica	60
Modos de Compilacion	60
Modo Estatico	60
Modo Dinamico	60
Script build_monolithic.sh	60
Proceso de Compilacion	60
Paso 1: Compilar Bridge C++	60
Paso 2: Compilar Pascal	61
Paso 3: Linkeo Final	61
Verificacion	61
Programas de Prueba	63
1. TestGecodeBridge	63
2. GecodeInfo	63
3. TestComplejo	64
Problema 1: SEND + MORE = MONEY	64
Problema 2: Cuadrado Magico 3x3	65
4. TestIntervalos	65
Ejemplos de Uso	67
Ejemplo 1: Resolver CSP Simple	67
Ejemplo 2: Todas las Soluciones	67
Ejemplo 3: Exploracion del Espacio	68
Resumen de la Integracion	70
Ventajas de esta Arquitectura	70
Limitaciones	70
Flujo Completo	70

Introduccion

?Que es GeCode CSP Pipeline?

GeCode CSP Pipeline es un sistema completo de procesamiento para problemas de **Constraint Satisfaction Problems (CSP)** que integra herramientas de validacion, analisis y resolucion mediante Gecode.

El pipeline procesa problemas definidos en formato JSON, pasando por multiples etapas de verificacion y optimizacion, hasta obtener soluciones completas usando el solver Gecode.

Proposito del Proyecto

Este sistema fue disenado para:

1. **Validar** especificaciones de problemas CSP en formato JSON
2. **Analizar** y transformar expresiones en grafos AST
3. **Verificar** la consistencia de restricciones
4. **Propagar** restricciones usando algoritmos como AC-3
5. **Resolver** problemas CSP completos con Gecode
6. **Persistir** resultados intermedios opcionalmente en SQLite

Caracteristicas Principales

? Validacion y Analisis

- **SyntaxChecker**: Validacion robusta de sintaxis JSON
- **JsonToGraph**: Construccin de AST y grafos de restricciones
- **FunctionChecker**: Verificacion de funciones definidas por el usuario

? Propagacion de Restricciones

- **FwdConsistency**: Implementacion del algoritmo AC-3 (Arc Consistency)
- **BwdConsistency**: Analisis de intervalos y proyeccion inversa
- **ForwardChain**: Encadenamiento hacia adelante para inferencia logica

? Resolucion CSP

- **TestGecodeBridge**: Puente a Gecode para resolucion completa
- **Soporte de 4 tipos**: integer, float, logic, set
- **Multiples soluciones**: Búsqueda exhaustiva o primera solucion

? Persistencia y Utilidades

- **JsonSink**: Almacenamiento en SQLite
- **JsonSource**: Recuperacion desde SQLite
- **CSPEval**: Evaluador de expresiones

? Compilacion Flexible

- **Pipeline tools**: Compilacion sin Gecode
- **Monolitica**: Ejecutables standalone de ~8-12 MB

- **Estatica/Dinamica:** Soporte para linkeo estatico o dinamico de Gecode

Tipos de Datos Soportados

Tipo	Descripcion	Formato JSON
integer	Numeros enteros	{"domain": [1, 100], "value": 10}
float	Numeros de punto flotante	{"domain": [0.0, 50.0], "value": 15.5}
logic	Valores booleanos	{"domain": [true, false], "value": true}
set	Conjuntos de valores	{"domain": {"miembros": ["A", "B"]}, "value": "A"}

Operadores Soportados

Aritmeticos

- + Suma
- - Resta
- * Multiplicacion
- / Division

Logicos

- AND Conjuncion logica
- OR Disyuncion logica
- NOT Negacion
- IMPLICA Implicacion

Relacionales

- = Igualdad
- <> Desigualdad
- < Menor que
- > Mayor que
- <= Menor o igual
- >= Mayor o igual

Conjuntos

- UNION Union de conjuntos
- INTERSECT Interseccion
- DIFFERENCE Diferencia
- SUBSET Subconjunto
- CARDINALITY Numero de elementos
- IN Pertenencia

Funciones Matematicas

El sistema incluye la biblioteca **MiniMath** con funciones:

- `abs(x)` - Valor absoluto
- `sqrt(x)` - Raiz cuadrada
- `sqr(x)` - Cuadrado
- `sin(x)`, `cos(x)`, `tan(x)` - Funciones trigonometricas
- `ln(x)` - Logaritmo natural
- `exp(x)` - Exponencial
- `pow(x, y)` - Potencia

Compilacion e Instalacion

Requisitos del Sistema

Para Herramientas del Pipeline

- **Free Pascal Compiler** (fpc) ≥ 3.0
- **gcc** ≥ 7.0
- **make**
- **SQLite3** (libsqlite3-dev)

Para Integracion con Gecode

- **g++** ≥ 7.0
- **Gecode** 6.x (estatico o dinamico)
- **Free Pascal Compiler** (fpc) ≥ 3.0

Instalacion de Dependencias

Ubuntu/Debian

```
# Herramientas basicas
sudo apt install build-essential fpc gcc make libsqlite3-dev

# Para Gecode (version dinamica del sistema)
sudo apt install libgecode-dev

# Para compilar PDFs (opcional)
sudo apt install pandoc texlive-latex-base texlive-fonts-recommended texlive-latex-extra
```

Fedora/RHEL

```
# Herramientas basicas
sudo dnf install gcc fpc make sqlite-devel

# Para compilar PDFs (opcional)
sudo dnf install pandoc texlive-scheme-basic
```

Instalacion de Gecode Estatico

Para crear ejecutables verdaderamente standalone, se recomienda compilar Gecode estatico:

```
cd /tmp
wget https://github.com/Gecode/gecode/archive/refs/tags/release-6.3.0.tar.gz
tar xzf release-6.3.0.tar.gz
cd gecode-release-6.3.0

./configure \
  --prefix=$HOME/gecode-static \
  --disable-shared \
```

```

--enable-static \
--disable-examples \
--disable-qt \
--disable-gist

make -j$(nproc)
make install

# Configurar variable de entorno
export GECODE_HOME=$HOME/gecode-static
echo "export GECODE_HOME=$HOME/gecode-static" >> ~/.bashrc

```

Compilacion del Proyecto

Compilacion Completa

```

# Clonar o descargar el proyecto
cd gecode

# Compilar todo (pipeline + gecode)
make

# Verificar ejecutables generados
ls -lh bin/

```

Compilacion Selectiva

```

# Solo herramientas del pipeline (sin Gecode)
make pipeline

# Solo herramientas Gecode
make gecode

# Limpieza
make clean      # Solo objetos
make distclean  # Todo

```

Compilacion Monolitica

Para crear ejecutables totalmente standalone:

```
./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

Esto genera un ejecutable que incluye:

- ? Todo el codigo Pascal compilado
- ? Puente C++ a Gecode
- ? Gecode linkeado estaticamente
- ? Sin dependencias de libgecode*.so

- ? Tamano: ~8-12 MB

Verificacion de la Instalacion

```
# Probar herramientas del pipeline
./bin/SyntaxChecker ejemplos/Json_input_1.json
./bin/JsonToGraph ejemplos/Json_input_1.json
./bin/FwdConsistency graph.json

# Probar integracion Gecode
./bin/TestGecodeBridge graph.json

# Ejecutar pipeline completo
./scripts/pipeline.sh ejemplos/Json_input_1.json
```

Si todos los comandos ejecutan sin errores, la instalacion fue exitosa.

Inicio Rapido

Ejemplo Basico

1. Crear un archivo JSON de entrada

Crear `mi_problema.json`:

```
{
  "precision": 2,
  "variables": [
    {
      "nombre": "x",
      "tipo": "integer",
      "domain": [1, 10],
      "value": null
    },
    {
      "nombre": "y",
      "tipo": "integer",
      "domain": [1, 10],
      "value": null
    }
  ],
  "expresiones": [
    "x + y = 10",
    "x > y"
  ]
}
```

2. Ejecutar el pipeline

```
./scripts/pipeline.sh mi_problema.json
```

3. Revisar resultados

El pipeline ejecutara todas las etapas y mostrara:

- ? Validacion de sintaxis
- ? Construccion de AST
- ? Verificacion de funciones
- ? Propagacion de restricciones
- ? Soluciones encontradas por Gecode

Pipeline con Persistencia

```
# Persistir cada etapa en SQLite
./scripts/pipeline.sh --db resultados.db --tag prueba_1 mi_problema.json

# Procesar multiples archivos
```

```
./scripts/pipeline.sh --db resultados.db ejemplos/*.json
```

```
# Recuperar resultados de una etapa
```

```
./bin/JsonSource resultados.db prueba_1_syntax
```

```
./bin/JsonSource resultados.db prueba_1_graph
```

```
./bin/JsonSource resultados.db prueba_1_csp
```

Uso Individual de Componentes

```
# 1. Validar sintaxis
```

```
./bin/SyntaxChecker ejemplos/Json_input_1.json
```

```
# 2. Construir grafo AST
```

```
./bin/JsonToGraph ejemplos/Json_input_1.json > graph.json
```

```
# 3. Verificar funciones
```

```
./bin/FunctionChecker graph.json > graph_checked.json
```

```
# 4. Propagacion forward
```

```
./bin/FwdConsistency graph_checked.json > graph_fwd.json
```

```
# 5. Propagacion backward
```

```
./bin/BwdConsistency graph_fwd.json > graph_bwd.json
```

```
# 6. Resolucion con Gecode
```

```
./bin/TestGecodeBridge graph_bwd.json > solucion.json
```

Explorar Ejemplos

El directorio ejemplos/ contiene multiples archivos de prueba:

```
# Listar ejemplos disponibles
```

```
ls -l ejemplos/Json_input_*.json
```

```
# Ver contenido de un ejemplo
```

```
cat ejemplos/Json_input_1.json
```

```
# Ejecutar un ejemplo
```

```
./scripts/pipeline.sh ejemplos/Json_input_2.json
```

```
# Ejecutar todos los ejemplos
```

```
for f in ejemplos/Json_input_*.json; do
```

```
    echo "=== Procesando $f ==="
```

```
    ./scripts/pipeline.sh "$f"
```

```
    echo
```

```
done
```

Proximos Pasos

1. Leer **02_arquitectura.md** para entender el diseno del sistema
2. Leer **03_componentes_pipeline.md** para detalles de cada componente
3. Leer **04_integracion_gecode.md** para entender el puente a Gecode
4. Revisar los reportes de pruebas en **docs/reporte_*.md**

Nota: Esta es la version inicial del manual. Para informacion detallada sobre cada componente, consultar los documentos especificos en **docs/**.

Vision General de la Arquitectura

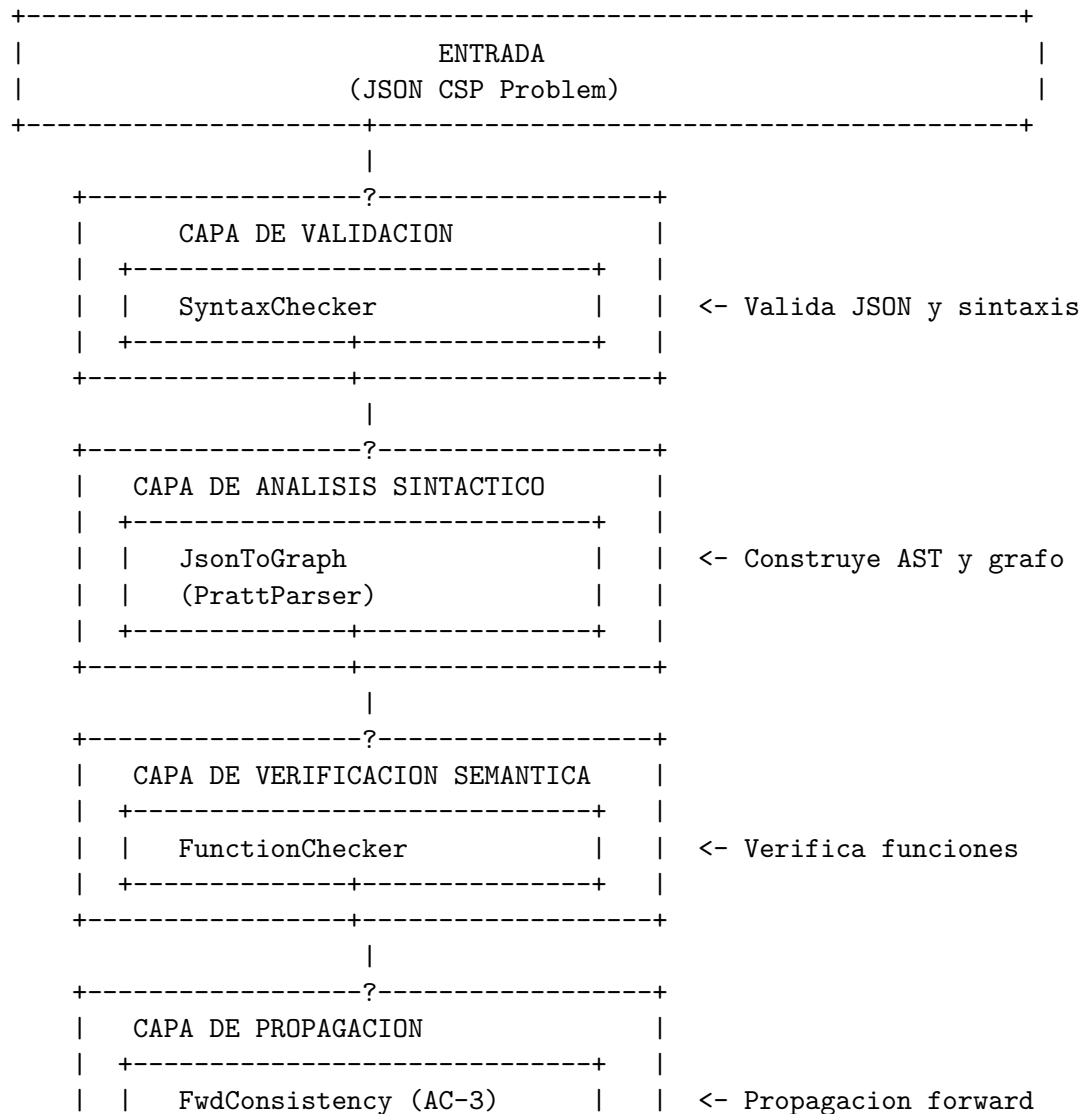
Filosofia de Diseno

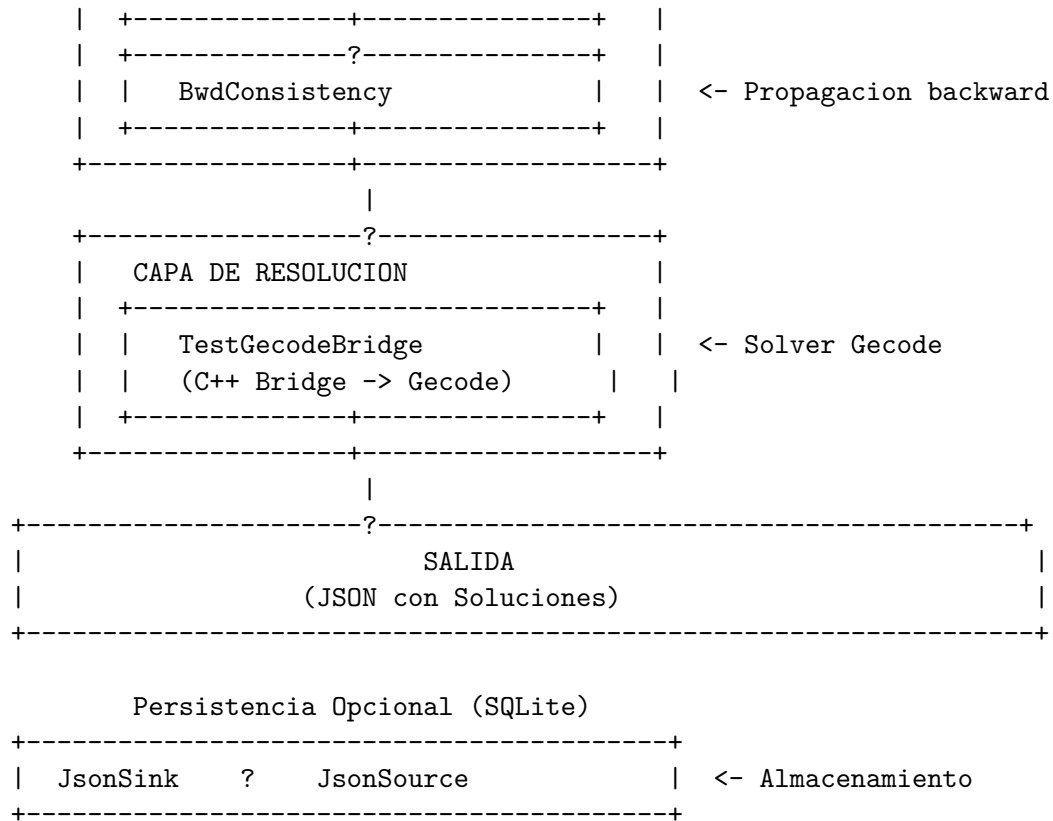
El GeCode CSP Pipeline sigue una arquitectura de **pipeline modular** donde cada componente tiene una responsabilidad unica y bien definida. Los datos fluyen secuencialmente a traves de etapas de transformacion y verificacion.

Principios de Diseno

1. **Separacion de Responsabilidades:** Cada herramienta hace una cosa y la hace bien
2. **Formato JSON:** Comunicacion entre componentes mediante JSON estructurado
3. **Composabilidad:** Las herramientas pueden usarse independientemente o en pipeline
4. **Persistencia Opcional:** SQLite permite almacenar/recuperar estados intermedios
5. **Compilacion Flexible:** Soporte para compilacion modular o monolitica

Arquitectura de Alto Nivel





Componentes del Sistema

1. SyntaxChecker

Proposito: Validar la sintaxis del archivo JSON de entrada

Entrada: Archivo JSON **Salida:** JSON validado o mensaje de error

Funciones:

- Verificar estructura JSON valida
- Validar campos obligatorios (precision, variables, expresiones)
- Verificar tipos de datos
- Reportar errores de sintaxis

Tecnologias: Pascal + MiniJSON

2. JsonToGraph

Proposito: Construir el AST y grafo de restricciones

Entrada: JSON validado **Salida:** Grafo JSON con AST de expresiones

Funciones:

- Parsear expresiones con PrattParser
- Construir arbol de sintaxis abstracta
- Generar grafo de dependencias
- Identificar nodos y aristas

Tecnologias: Pascal + PrattParser + ExpressionAST

3. FunctionChecker

Proposito: Verificar funciones definidas por el usuario

Entrada: Grafo JSON **Salida:** Grafo verificado

Funciones:

- Validar definiciones de funciones
- Verificar parametros y tipos
- Comprobar referencias validas
- Marcar funciones validas/invalidas

Tecnologias: Pascal + UCSPJson

4. FwdConsistency

Proposito: Propagacion de restricciones hacia adelante (AC-3)

Entrada: Grafo verificado **Salida:** Grafo con dominios reducidos

Funciones:

- Implementar algoritmo AC-3
- Reducir dominios de variables
- Propagar restricciones forward
- Detectar inconsistencias tempranas

Tecnologias: Pascal + MiniMath + AC-3 Algorithm

5. BwdConsistency

Proposito: Proyeccion inversa de restricciones

Entrada: Grafo con dominios reducidos **Salida:** Grafo con analisis de intervalos

Funciones:

- Analisis de intervalos
- Proyeccion inversa de restricciones
- Refinamiento de dominios
- Propagacion backward

Tecnologias: Pascal + MiniMath + Interval Analysis

6. TestGecodeBridge

Proposito: Resolucion CSP completa con Gecode

Entrada: Grafo optimizado **Salida:** JSON con soluciones

Funciones:

- Traducir CSP a modelo Gecode
- Configurar variables y restricciones
- Ejecutar busqueda
- Retornar soluciones encontradas

Tecnologias: Pascal + C++ Bridge + Gecode Solver

Herramientas Auxiliares

CSPEval

Evaluador de expresiones CSP con calculo de resultados

ForwardChain

Encadenamiento hacia adelante para inferencia logica

JsonSink / JsonSource

Persistencia y recuperacion desde SQLite

Flujo de Datos

Formato JSON de Entrada

```
{
  "precision": 2,
  "variables": [
    {
      "nombre": "x",
      "tipo": "integer",
      "domain": [1, 100],
      "value": 10
    },
    {
      "nombre": "y",
      "tipo": "float",
      "domain": [0.0, 50.0],
      "value": 15.5
    }
  ],
  "expresiones": [
    "x + y * 2",
    "x > 5"
  ],
  "funciones": [
    {
      "nombre": "miFuncion",
      "parametros": ["a", "b"],
      "salida": "a + b * 2"
    }
  ]
}
```

Transformaciones por Etapa

Etapas: SyntaxChecker -> JSON Validado

```
{
  "valido": true,
  "errores": [],
  "precision": 2,
  "variables": [...],
  "expresiones": [...]
}
```

Etapla 2: JsonToGraph -> Grafo con AST

```
{
  "precision": 2,
  "variables": [...],
  "expresiones": [
    {
      "texto": "x + y * 2",
      "ast": {
        "tipo": "BINOP",
        "op": "+",
        "izq": {"tipo": "IDENT", "nombre": "x"},
        "der": {
          "tipo": "BINOP",
          "op": "*",
          "izq": {"tipo": "IDENT", "nombre": "y"},
          "der": {"tipo": "LITERAL", "valor": 2}
        }
      }
    }
  ],
  "grafo": {
    "nodos": [...],
    "aristas": [...]
  }
}
```

Etapla 3: FunctionChecker -> Grafo Verificado

```
{
  "funciones_verificadas": true,
  "funciones": [
    {
      "nombre": "miFuncion",
      "valida": true,
      "mensaje": "OK"
    }
  ],
  ...
}
```

Etapla 4-5: Propagacion -> Dominios Reducidos

```
{
  "variables": [
    {
      "nombre": "x",

```

```

    "domain_original": [1, 100],
    "domain_reducido": [6, 100]  <- Despues de x > 5
  }
],
...
}

```

Etapla 6: TestGecodeBridge -> Soluciones

```

{
  "num_soluciones": 3,
  "soluciones": [
    {
      "x": 10,
      "y": 15.5
    },
    {
      "x": 11,
      "y": 14.25
    },
    {
      "x": 12,
      "y": 13.0
    }
  ],
  "estadisticas": {
    "tiempo_ms": 42,
    "nodos_explorados": 156
  }
}

```

Arquitectura de Persistencia

SQLite como Almacenamiento Intermedio

El sistema permite **persistir cada etapa del pipeline** en una base de datos SQLite mediante las herramientas **JsonSink** y **JsonSource**.

Esquema de Base de Datos

```
CREATE TABLE json_storage (  
    tag TEXT PRIMARY KEY,  
    timestamp INTEGER,  
    json_data TEXT  
);
```

Flujo con Persistencia

Input JSON

?

```
SyntaxChecker -> JsonSink(db, "problema1_syntax")  
                  ?
```

```
                  JsonSource(db, "problema1_syntax")
```

?

```
JsonToGraph -> JsonSink(db, "problema1_graph")
```

?

```
                  JsonSource(db, "problema1_graph")
```

?

```
FunctionChecker -> JsonSink(db, "problema1_checked")
```

?

...

Ventajas de la Persistencia

1. **Debugging:** Inspeccionar estado en cualquier etapa
2. **Reproducibilidad:** Re-ejecutar desde cualquier punto
3. **Analisis:** Comparar resultados de multiples ejecuciones
4. **Cacheo:** Evitar re-computar etapas costosas

Uso

```
# Almacenar JSON  
echo '{"data": "value"}' | ./bin/JsonSink db.sqlite mi_tag  
  
# Recuperar JSON  
./bin/JsonSource db.sqlite mi_tag  
  
# Pipeline completo con persistencia  
./scripts/pipeline.sh --db runs.db --tag test_1 input.json
```

Integracion de Lenguajes

Pascal + C/C++ + Gecode

El proyecto integra **tres lenguajes** de forma eficiente:

1. Pascal (Free Pascal)

Usado para: - Herramientas del pipeline - Parsing y validacion - Logica de negocio - Interfaz de linea de comandos

Ventajas: - Compilacion rapida - Ejecutables eficientes - Sintaxis clara y legible - Excelente para procesamiento de texto

2. C (Biblioteca MiniMath)

Usado para: - Funciones matematicas (sin, cos, exp, ln) - Operaciones de intervalos - Calculos de precision

Integracion:

```
{ $LINKLIB minimath }
{ $L minimath_trig.o }
{ $L minimath_exp.o }
{ $L minimath_interval.o }

function c_sin(x: cdouble): cdouble; cdecl; external;
function c_cos(x: cdouble): cdouble; cdecl; external;
```

3. C++ (Gecode Bridge)

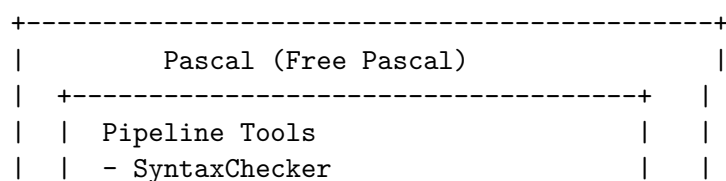
Usado para: - Interfaz a Gecode - Resolucion CSP - Busqueda y optimizacion

Integracion Pascal -> C++:

```
// UGecodeBridge.pas
function csp_create(vars: PVariable; n: cint): pointer; cdecl; external;
function csp_solve(model: pointer; sol: PSolution): cint; cdecl; external;

// gecode_bridge.cpp
extern "C" {
    void* csp_create(Variable* vars, int n) { ... }
    int csp_solve(void* model, Solution* sol) { ... }
}
```

Diagrama de Integracion



- JsonToGraph	
- FwdConsistency	
+-----+	
{\$LINKLIB}	
?	
+-----+	
MiniMath (C)	
- minimath_trig.o	
- minimath_exp.o	
- minimath_interval.o	
+-----+	
+-----+	
+-----+	
Pascal + C++ Bridge	
+-----+	
TestGecodeBridge.pas	
(Programa Pascal)	
+-----+	
external cdecl	
?	
+-----+	
UGecodeBridge.pas	
(Interface Unit)	
+-----+	
FFI	
?	
+-----+	
gecode_bridge.cpp	
(C API Wrapper)	
+-----+	
?	
+-----+	
Gecode (C++ Library)	
- IntVarArray	
- Constraints	
- Search	
+-----+	
+-----+	

Sistema de Compilacion

Makefile Modular

El Makefile permite compilar componentes de forma **selectiva**:

```
all: pipeline gecode

pipeline: minimath json_tools propagation_tools persistence_tools

gecode: gecode_bridge gecode_tools

minimath:
    gcc -c src/minimath_*.c

json_tools:
    fpc src/JsonToGraph.pas
    fpc src/SyntaxChecker.pas

gecode_bridge:
    g++ -c src/gecode_bridge.cpp -I$(GECODE_HOME)/include

gecode_tools:
    ./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

Compilacion Monolitica

El script build_monolithic.sh genera **ejecutables standalone**:

```
#!/bin/bash
# 1. Compilar bridge C++
g++ -c gecode_bridge.cpp

# 2. Compilar Pascal con linkeo explicito
fpc TestGecodeBridge.pas \
    -FU obj/ \
    -k'-L/path/to/gecode/lib' \
    -k'-lgecodekernel' \
    -k'-lgecodesupport' \
    -k gecode_bridge.o

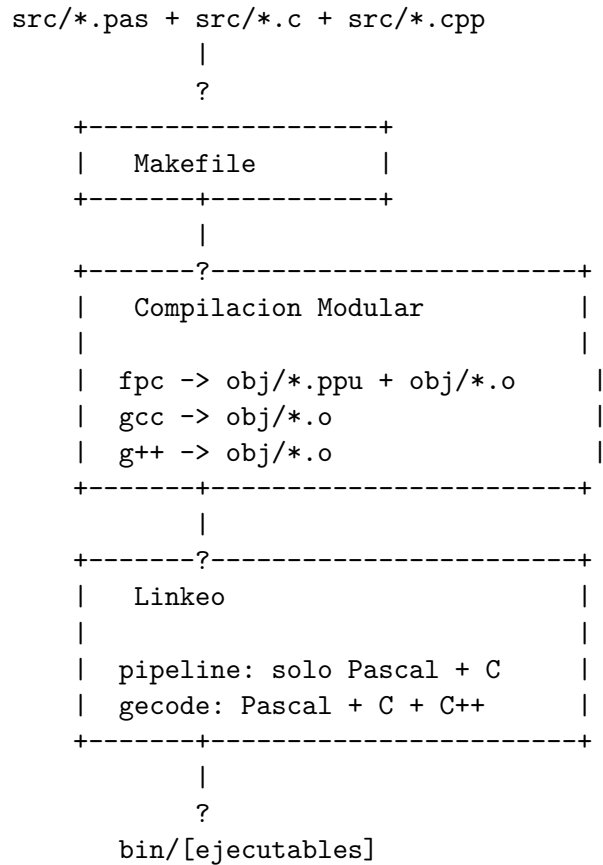
# Resultado: ejecutable standalone de ~8-12 MB
```

Modos de Linkeo

1. **Estatico**: Si existe \$GECODE_HOME/lib/*.a
 - Linkea libgecode*.a estaticamente
 - No requiere libgecode*.so en runtime
 - Ejecutable mas grande (~12 MB)
2. **Dinamico**: Si solo hay /usr/lib/libgecode*.so

- Linkea dinamicamente
- Requiere libgecode*.so instaladas
- Ejecutable mas pequeno (~500 KB)

Diagrama de Compilacion



Proximo: Leer `03_componentes_pipeline.md` para detalles tecnicos de cada componente.

Componentes del Pipeline

El pipeline de GeCode CSP esta compuesto por **9 componentes principales** que transforman y procesan problemas CSP desde la entrada JSON hasta la resolucion completa. Cada componente tiene una responsabilidad especifica y bien definida.

Flujo General

JSON Input

- ?
- [1] SyntaxChecker -> Validacion de sintaxis y semantica
- ?
- [2] JsonToGraph -> Construccion de AST y grafo
- ?
- [3] FunctionChecker -> Verificacion de funciones
- ?
- [4] FwdConsistency -> Propagacion forward (AC-3)
- ?
- [5] BwdConsistency -> Propagacion backward (HC4)
- ?
- [6] CSPEval -> Evaluacion rapida
- ?
- [7] ForwardChain -> Encadenamiento iterativo
- ?
- [8] TestGecodeBridge -> Resolucion con Gecode
- ?

Solutions Output

Persistencia:

- [9] JsonSink/JsonSource -> SQLite storage

1. SyntaxChecker

Proposito

Validar la **sintaxis y coherencia semantica** de un archivo JSON de sistema CSP antes de procesarlo con el pipeline. Actua como el primer filtro de calidad para detectar errores tempranos.

Entrada/Salida

Entrada: Archivo JSON con campos **variables**, **expressions**, **functions**.

Salida: JSON con resultado de validacion:

```
{
  "status": "ok"
}
```

O en caso de errores:

```
{
  "status": "error",
  "errors": [
    {
      "rule": "nombre_regla",
      "msg": "descripcion del error"
    }
  ]
}
```

Reglas de Validacion

El SyntaxChecker verifica las siguientes reglas:

1. **Campos obligatorios presentes:** **variables**, **expressions**
2. **Variables referenciadas existen:** Todas las variables usadas en constraints estan declaradas
3. **Funciones usadas estan declaradas:** Referencias a funciones resuelven correctamente
4. **Referencias validas:** Variables y funciones se resuelven sin ambigüedades
5. **Sintaxis parseable:** Cada constraint tiene sintaxis valida
6. **Coherencia de dominios:** El campo **value** es subconjunto del **domain**
7. **Aridad correcta:** Funciones user-defined tienen el numero correcto de parametros
8. **No sobrescritura:** No se sobrescriben built-ins del motor (**abs**, **dist**, etc.)

Codigos de Salida

- 0 = validacion exitosa
- 1 = errores de validacion encontrados

Uso

```
# Validar archivo
./bin/SyntaxChecker sistema.json
```

En el pipeline

```
./bin/SyntaxChecker sistema.json && ./bin/JsonToGraph sistema.json | ...
```

Ejemplo

```
$ ./bin/SyntaxChecker ejemplos/Json_input_1.json
{
  "status": "ok"
}

$ echo $?
0
```

Tecnologias

- **Pascal + MiniJSON** para parsing JSON
- Validacion semantica custom

Posicion en el Pipeline

ETAPA 1 – Siempre primera antes de JsonToGraph.

2. JsonToGraph

Proposito

Convierte un JSON de sistema CSP en un **grafo de restricciones con AST**. Es la etapa central que transforma expresiones de texto en una estructura de datos procesable por el resto del pipeline.

Entrada/Salida

Entrada: Archivo JSON con **variables**, **expressions**, **functions**.

Salida: JSON con cuatro secciones: - **variables** – variables con id asignado, tipo y dominio - **functions** – funciones user-defined declaradas - **constraints** – cada constraint con su AST (nodos tipados) y referencias a variables/funciones usadas - **adjacency** – indice inverso: que constraints afectan a cada variable

Algoritmo Principal

Utiliza un **Pratt Parser** para construir el arbol de sintaxis abstracta de cada expresion:

1. **Tokenizacion:** Divide la expresion en tokens
2. **Parsing por precedencia:** Construye AST respetando precedencia de operadores
3. **Construccion de grafo:** Genera nodos y aristas del grafo de dependencias
4. **Indexacion:** Crea indice inverso de que constraints afectan a cada variable

Tipos de Nodo AST

El parser genera los siguientes tipos de nodos:

Nodos de Valor

- **Variable** – referencia a variable
- **Number** – literal numerico
- **Set** – conjunto literal
- **Interval** – intervalo [a, b]

Nodos Relacionales

- **Equals** – operador =
- **NotEquals** – operador <>
- **Less** – operador <
- **Greater** – operador >
- **LessEq** – operador <=
- **GreaterEq** – operador >=

Nodos Logicos

- **And** – operador AND
- **Or** – operador OR
- **Not** – operador NOT

Nodos Aritmeticos

- Add – operador +
- Subtract – operador -
- Multiply – operador *
- Divide – operador /
- Negate – negacion unaria

Nodos Especiales

- FunctionCall – llamada a funcion
- In – operador de pertenencia IN

Uso

```
# Salida a stdout
./bin/JsonToGraph sistema.json

# Salida a archivo
./bin/JsonToGraph sistema.json salida.json

# En el pipeline
./bin/JsonToGraph sistema.json | ./bin/FwdConsistency
```

Ejemplo de Transformacion

Entrada:

```
{
  "variables": [
    {"nombre": "x", "tipo": "integer", "domain": [1, 10]}
  ],
  "expresiones": ["x + y * 2"]
}
```

Salida (AST):

```
{
  "ast": {
    "tipo": "BINOP",
    "op": "+",
    "izq": {"tipo": "IDENT", "nombre": "x"},
    "der": {
      "tipo": "BINOP",
      "op": "*",
      "izq": {"tipo": "IDENT", "nombre": "y"},
      "der": {"tipo": "LITERAL", "valor": 2}
    }
  }
}
```


Tecnologias

- **Pascal + PrattParser + ExpressionAST**
- Parsing por precedencia de operadores
- Construccion de grafo dirigido aciclico

Posicion en el Pipeline

ETAPA 2 – Despues de SyntaxChecker, antes de todo lo demas.

3. FunctionChecker

Proposito

Verifica que los **archivos objeto** (.so / .o) de las funciones user-defined referenciadas en el grafo existan en los directorios de busqueda antes de intentar ejecutar el pipeline.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con resultado:

```
{
  "status": "ok"|"missing"|"error",
  "checked": N,
  "found": N,
  "missing": N,
  "functions": [
    {
      "name": "miFuncion",
      "object": "func_miFuncion.so",
      "path": "./lib/func_miFuncion.so",
      "found": true
    }
  ]
}
```

Algoritmo de Busqueda

1. **Extraer funciones:** Lee la seccion `functions` del grafo JSON
2. **Buscar objetos:** Para cada funcion, busca `func_<nombre>.so` o `func_<nombre>.o` en:
 - `.` (directorio actual)
 - `./lib`
 - `./obj`
 - `/usr/local/lib/csp`
 - Directorios adicionales con `--path`
3. **Reportar resultados:** Marca cada funcion como encontrada o faltante

Directorios de Busqueda

Por defecto: `../lib:./obj:/usr/local/lib/csp`

Puede personalizarse:

```
./bin/FunctionChecker grafo.json --path ./lib:./obj:/usr/local/lib/csp
```

Codigos de Salida

- 0 = todos los objetos encontrados
- 1 = algun objeto faltante (puede continuar con advertencia)

- 2 = error fatal (no se puede continuar)

Uso

```
# Verificacion basica
./bin/FunctionChecker grafo.json

# Con directorios personalizados
./bin/FunctionChecker grafo.json --path ./custom_libs:./obj

# En el pipeline
./bin/JsonToGraph sistema.json | ./bin/FunctionChecker
```

Notas

- Solo relevante si el sistema usa funciones user-defined
- Si no hay funciones, el resultado es siempre “ok”
- Los objetos no se cargan, solo se verifica su existencia

Tecnologias

- **Pascal + UCSPJson**
- Búsqueda de archivos en filesystem

Posicion en el Pipeline

ETAPA 3 – Despues de JsonToGraph, antes de FwdConsistency.

4. FwdConsistency

Proposito

Aplica **consistencia de arco hacia adelante (AC-3)** al grafo de restricciones. Propaga cada constraint desde las variables conocidas hacia las desconocidas, reduciendo dominios progresivamente.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** - “arc_consistent” | “inconsistent” - **queue_ops** - operaciones de cola AC-3 - **firings** - reglas disparadas - **steps** - array de pasos con cada reduccion de dominio - **variables** - dominios finales de todas las variables

Algoritmo AC-3

Arc Consistency Algorithm 3 (AC-3):

1. Inicializar cola Q con todos los arcos (X, C) donde X es variable, C es constraint que involucra a X
2. Mientras Q no este vacia:
 - a. Extraer arco (X, C) de Q
 - b. Si Revise(X, C) reduce el dominio de X:
 - Si domain(X) esta vacio -> INCONSISTENT
 - Agregar a Q todos los arcos (Y, C') donde Y depende de X
3. Si ningun dominio quedo vacio -> ARC_CONSISTENT

Revise(X, C):

Para cada valor v en domain(X):

 Evaluar constraint C con X=v

 Si C no se puede satisfacer con ningun valor de otras variables:

 Eliminar v de domain(X)

Retornar true si algun valor fue eliminado

Ejemplo de Paso

```
{
  "steps": [
    {
      "var": "X",
      "constraint": "X > 5",
      "before": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
      "after": [6, 7, 8, 9, 10]
    }
  ]
}
```

Uso

```
# Salida a stdout
./bin/FwdConsistency grafo.json

# Salida a archivo
./bin/FwdConsistency grafo.json salida.json

# Con persistencia
./bin/FwdConsistency grafo.json | ./bin/JsonSink runs.db fwd
```

Tecnologias

- **Pascal + MiniMath + AC-3 Algorithm**
- Evaluacion de restricciones con aritmetica de intervalos

Posicion en el Pipeline

ETAPA 4 – Despues de FunctionChecker. Complementa a BwdConsistency: ambos se aplican al mismo grafo.

5. BwdConsistency

Proposito

Aplica **consistencia de arco hacia atras (HC4-revise)** al grafo de restricciones. Para constraints del tipo $V = f(W)$, invierte f para restringir W en funcion del dominio reducido de V . Complementa a FwdConsistency.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con la misma estructura que FwdConsistency: - **status** - “arc_consistent” | “inconsistent” - **queue_ops** - operaciones de cola - **firings** - reglas disparadas - **steps** - reducciones de dominio (marcadas con “(bwd)”) - **variables** - dominios finales

Algoritmo HC4-Revise

Hull Consistency 4 (HC4) con proyeccion inversa:

Para cada constraint $V = f(X, Y, Z)$:

1. Forward: Calcular $\text{domain}(V) = f(\text{domain}(X), \text{domain}(Y), \text{domain}(Z))$
2. Backward: Para cada operando (ej. X):
 - a. Proyectar constraint: $X = f_{\text{inv}}(V, Y, Z)$
 - b. Calcular $\text{domain_nuevo}(X) = f_{\text{inv}}(\text{domain}(V), \text{domain}(Y), \text{domain}(Z))$
 - c. Intersectar: $\text{domain}(X) \leftarrow \text{domain}(X) \cap \text{domain_nuevo}(X)$
3. Iterar hasta punto fijo

Diferencia con FwdConsistency

Aspecto	FwdConsistency	BwdConsistency
Direccion	Izquierda -> Derecha	Derecha -> Izquierda
Algoritmo	AC-3 (Arc Consistency)	HC4 (Hull Consistency)
Estrategia	Evaluar constraint con valores	Proyectar funcion inversa
Ejemplo	$x + y = 10 \rightarrow$ reducir y dado x	$z = x + y \rightarrow$ reducir x, y dado z

Juntos cubren **mas reducciones de dominio** que cada uno por separado.

Ejemplo

Constraint: $z = x + y$

Forward:

$x \in [1, 5], y \in [2, 8]$
 $\rightarrow z \in [1+2, 5+8] = [3, 13]$

Backward:

$z \in [5, 10]$ (dominio reducido por otra restriccion)
 $\rightarrow x \in [5-8, 10-2] \cap [1, 5] = [-3, 8] \cap [1, 5] = [1, 5]$ (sin cambio)

-> $y \ ? \ [5-5, 10-1] \ ? \ [2, 8] = [0, 9] \ ? \ [2, 8] = [2, 8]$ (sin cambio)

Uso

```
# Salida a stdout
./bin/BwdConsistency grafo.json

# En paralelo con FwdConsistency
./bin/FwdConsistency grafo.json | ./bin/BwdConsistency

# Con persistencia
./bin/BwdConsistency grafo.json | ./bin/JsonSink runs.db bwd
```

Tecnologías

- **Pascal + MiniMath + Interval Analysis**
- Aritmetica de intervalos con proyeccion inversa

Posicion en el Pipeline

ETAPA 5 – En paralelo o secuencia con FwdConsistency.

6. CSPEval

Proposito

Evaluador CSP generico con propagacion iterativa de dominios. Maneja los cuatro tipos de variable del motor (numeric, integer, boolean, set) con operaciones de reduccion apropiadas para cada tipo.

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** – “ok” | “solved” | “contradiction” - **iterations** – iteraciones hasta punto fijo (max 200) - **variables** – dominios finales con flags

Tipos de Variable Soportados

Tipo	Dominio	Operaciones
numeric	Intervalo [lo, hi]	Aritmetica de punto flotante
integer	Enumerado de enteros	Operaciones discretas
boolean	{false, true}	Logica booleana
set	Enumerado de etiquetas	Operaciones de conjuntos

Algoritmo de Propagacion

1. Inicializar dominios de todas las variables
2. Repetir hasta punto fijo (max 200 iteraciones):
 - a. Para cada constraint:
 - Evaluar con dominios actuales
 - Calcular nuevo dominio resultante
 - Intersectar con dominio actual
 - b. Si algun dominio quedo vacio -> CONTRADICTION
 - c. Si todos los dominios tienen 1 solo valor -> SOLVED
 - d. Si ningun dominio cambio -> PUNTO FIJO (status: ok)
3. Retornar dominios finales

Ejemplo de Salida

```
{
  "status": "ok",
  "iterations": 5,
  "variables": [
    {
      "name": "X",
      "type": "integer",
      "domain": [6, 7, 8, 9, 10],
      "solved": false,
      "empty": false
    }
  ]
}
```



```

    },
    {
      "name": "Y",
      "type": "integer",
      "domain": [5],
      "solved": true,
      "empty": false
    }
  ]
}

```

Diferencia con FwdConsistency/ForwardChain

Aspecto	CSPEval	FwdConsistency	ForwardChain
Trazado	No genera steps	Genera steps detallados	Genera steps por iteracion
Velocidad	Mas rapido	Medio	Mas lento
Uso	Evaluacion rapida	Debugging/analisis	Debugging/inferencia

CSPEval produce **solo el resultado final** de dominios, sin trazar los pasos intermedios.

Uso

```

# Evaluacion rapida
./bin/CSPEval grafo.json

# Con persistencia
./bin/CSPEval grafo.json | ./bin/JsonSink runs.db cspeval

```

Tecnologias

- Pascal + MiniMath + Multi-domain evaluation
- Propagacion iterativa hasta punto fijo

Posicion en el Pipeline

Etapas de **evaluacion rapida** antes de pasar a TestGecodeBridge.

7. ForwardChain

Proposito

Encadenamiento hacia adelante iterativo: aplica propagacion de constraints en multiples pasadas hasta alcanzar un punto fijo (ningun dominio cambia) o detectar una contradiccion (dominio vacio).

Entrada/Salida

Entrada: JSON de grafo (salida de JsonToGraph).

Salida: JSON con: - **status** – “ok” | “solved” | “contradiction” - **iterations** – numero de pasadas realizadas - **steps** – array de pasos por iteracion - **variables** – dominios finales

Algoritmo de Encadenamiento

```
iteration = 1
REPEAT:
    changed = false
    FOR EACH constraint C:
        FOR EACH variable V in C:
            domain_old = domain(V)
            domain_new = evaluate_constraint(C, V)
            domain(V) = domain_old ? domain_new
            IF domain(V) != domain_old:
                Log step: {iteration, V, domain_old, domain_new}
                changed = true
            IF domain(V) is empty:
                RETURN "contradiction"
    iteration++
UNTIL NOT changed OR iteration > max_iterations

IF all variables have single value:
    RETURN "solved"
ELSE:
    RETURN "ok"
```

Estados de Retorno

- “solved” – todas las variables tienen exactamente un valor
- “contradiction” – algun dominio quedo vacio (sistema inconsistente)
- “ok” – reduccion parcial alcanzo punto fijo

Ejemplo de Steps

```
{
  "steps": [
    {
      "iteration": 1,
```

```

    "var": "X",
    "before": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "after": [6, 7, 8, 9, 10]
  },
  {
    "iteration": 2,
    "var": "Y",
    "before": [1, 2, 3, 4, 5],
    "after": [1, 2, 3, 4]
  }
]
}

```

Diferencia con FwdConsistency

Aspecto	FwdConsistency	ForwardChain
Estrategia	AC-3 con cola	Multiples pasadas completas
Eficiencia	Mas eficiente	Mas simple
Uso	Produccion	Trazado/debugging
Output	Steps ordenados por cola	Steps ordenados por iteracion

FwdConsistency aplica **AC-3 una sola vez con cola**. ForwardChain hace **multiples pasadas completas** sobre todos los constraints hasta punto fijo.

Uso

```

# Encadenamiento con trazado
./bin/ForwardChain grafo.json

# Con persistencia
./bin/ForwardChain grafo.json | ./bin/JsonSink runs.db forward

```

Tecnologias

- Pascal + MiniMath
- Evaluacion iterativa con punto fijo

Posicion en el Pipeline

Alternativa a FwdConsistency para sistemas con muchas iteraciones o cuando se requiere trazado detallado.

8. JsonSink

Proposito

Sumidero del pipeline: lee JSON de stdin y lo persiste en una base de datos SQLite, devolviendo por stdout un JSON de confirmacion encadenable.

Entrada/Salida

Entrada: JSON producido por cualquier etapa del pipeline (stdin).

Salida: JSON de confirmacion:

```
{
  "ok": true,
  "id": 42,
  "tag": "fwd",
  "status": "arc_consistent"
}
```

Esquema SQLite

```
CREATE TABLE runs (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  ts      TEXT,      -- timestamp ISO-8601 automatico
  tag     TEXT,      -- etiqueta de la etapa
  status  TEXT,      -- campo "status" extraido del JSON
  payload TEXT       -- JSON completo de la etapa
);
```

Funcionalidad

1. **Leer JSON** de stdin
2. **Extraer status** automaticamente del JSON
3. **Insertar registro** en tabla runs con:
 - timestamp automatico
 - tag (argumento 2)
 - status extraido
 - payload completo
4. **Emitir confirmacion** por stdout (encadenable)

Uso

```
# Almacenar JSON
echo '{"data": "value"}' | ./bin/JsonSink db.sqlite mi_tag

# Pipeline con persistencia
./bin/FwdConsistency grafo.json | ./bin/JsonSink pipeline.db fwd
```

```
# Cadena completa
```

```
./bin/JsonToGraph input.json | ./bin/JsonSink db.sqlite graph | ./bin/FwdConsistency
```

Notas

- El campo “**status**” se extrae automaticamente del JSON de entrada
- El **payload completo** se almacena para replay o auditoria
- Usa binding de **SQLite3** via cdecl; no requiere runtime adicional
- Para leer los datos persistidos, usar **JsonSource**

Tecnologias

- **Pascal + SQLite3 bindings**
- Insercion transaccional

Posicion en el Pipeline

Cierre de cualquier etapa cuando se desea persistencia.

9. JsonSource

Proposito

Fuente del pipeline: recupera un JSON persistido en SQLite y lo emite por stdout para continuar el pipeline desde un punto guardado.

Entrada/Salida

Entrada: Base de datos SQLite + filtros opcionales.

Salida: El payload JSON original del registro recuperado (stdout).

Modos de Recuperacion

```
# Ultimo registro insertado
./bin/JsonSource runs.db

# Ultimo registro con tag "fwd"
./bin/JsonSource runs.db fwd

# Registro con id=42 y tag="fwd"
./bin/JsonSource runs.db fwd 42
```

Casos de Uso Tipicos

1. Guardar y Reutilizar Grafo

```
# Guardar grafo
./bin/JsonToGraph sistema.json | ./bin/JsonSink pipeline.db graph

# Correr FwdConsistency y BwdConsistency desde el mismo grafo
./bin/JsonSource pipeline.db graph | ./bin/FwdConsistency | ./bin/JsonSink pipeline.db fwd
./bin/JsonSource pipeline.db graph | ./bin/BwdConsistency | ./bin/JsonSink pipeline.db bwd
```

2. Resolver CSP desde Grafo Guardado

```
# Resolver desde grafo persistido
./bin/JsonSource pipeline.db graph | ./bin/TestGecodeBridge
```

3. Recuperar Resultado Anterior

```
# Analizar resultado de etapa anterior
./bin/JsonSource pipeline.db csp 17 | python3 analizar.py
```

Notas

- Si no existe el tag o id, termina con **error en stderr** y **exit 1**
- Usa la misma binding minima de SQLite3 que JsonSink

- Permite **replay** y **debug** de cualquier etapa

Tecnologias

- **Pascal + SQLite3 bindings**
- Query transaccional

Posicion en el Pipeline

Inicio de cualquier etapa cuando el input viene de SQLite. Complemento de JsonSink – juntos implementan **memoria persistente**.

Resumen de Componentes

Tabla Comparativa

Componente	Tipo	Entrada	Salida	Proposito Principal
SyntaxChecker	Validacion	JSON raw	JSON validado	Verificar sintaxis y semantica
JsonToGraph	Transformacion	JSON validado	Grafo + AST	Construir estructura procesable
FunctionChecker	Verificacion	Grafo JSON	Reporte	Verificar objetos de funciones
FwdConsistency	Propagacion	Grafo JSON	Grafo + dominios	AC-3 forward
BwdConsistency	Propagacion	Grafo JSON	Grafo + dominios	HC4 backward
CSPEval	Evaluacion	Grafo JSON	Dominios finales	Evaluacion rapida
ForwardChain	Propagacion	Grafo JSON	Grafo + steps	Encadenamiento iterativo
JsonSink	Persistencia	JSON (stdin)	Confirmacion	Almacenar en SQLite
JsonSource	Persistencia	SQLite + filtros	JSON (stdout)	Recuperar desde SQLite

Pipelines Tipicos

Pipeline Completo con Propagacion

```
./bin/SyntaxChecker input.json && \  
./bin/JsonToGraph input.json | \  
./bin/FunctionChecker | \  
./bin/FwdConsistency | \  
./bin/BwdConsistency | \  
./bin/TestGecodeBridge
```

Pipeline con Persistencia

```
./bin/JsonToGraph input.json | ./bin/JsonSink db.sqlite graph  
./bin/JsonSource db.sqlite graph | ./bin/FwdConsistency | ./bin/JsonSink db.sqlite fwd  
./bin/JsonSource db.sqlite fwd | ./bin/TestGecodeBridge | ./bin/JsonSink db.sqlite csp
```

Pipeline de Evaluacion Rapida

```
./bin/SyntaxChecker input.json && \  
./bin/JsonToGraph input.json | \  
./bin/CSPEval | \  
./bin/TestGecodeBridge
```


Proximo: Leer `04_integracion_gecode.md` para entender el puente Pascal/C++/Gecode.

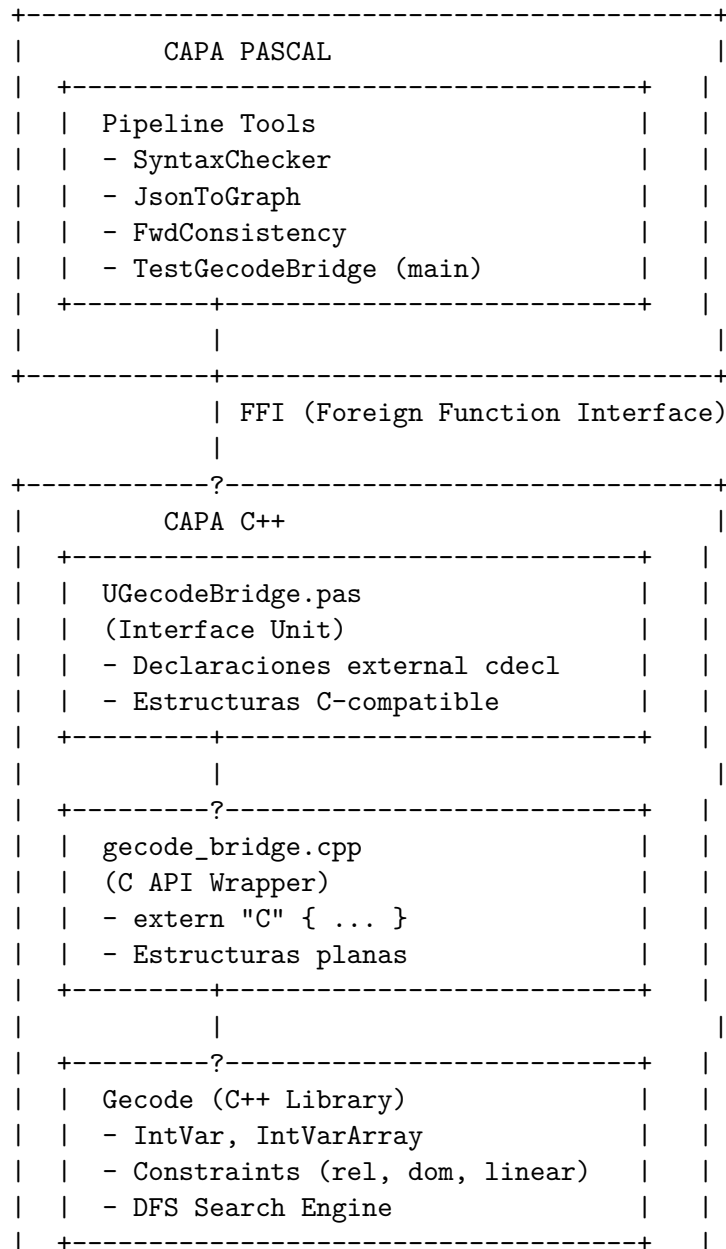
Introduccion a la Integracion

El GeCode CSP Pipeline integra **tres lenguajes** de forma eficiente para combinar las fortalezas de cada uno:

- **Pascal (Free Pascal)**: Logica de negocio, parsing, pipeline
- **C (MiniMath)**: Funciones matematicas, aritmetica de intervalos
- **C++ (Gecode)**: Resolucion CSP con programacion por restricciones

Esta integracion multi-lenguaje permite crear **ejecutables monoliticos** totalmente standalone que no requieren dependencias externas.

Vision General



+-----+

Arquitectura del Puente Pascal/C++

Componentes del Puente

1. UGecodeBridge.pas (Lado Pascal)

Proposito: Unit Pascal que declara las funciones C++ exportadas y proporciona helpers para construir estructuras.

Ubicacion: /home/rodo/gecode/src/UGecodeBridge.pas

Caracteristicas: - Usa `{ $PACKRECORDS C }` para alinear estructuras igual que C/C++ - Usa `{ $L ../obj/gecode_bridge.o }` para linkear el objeto C++ - Declara funciones con `cdecl; external;` (C calling convention) - Proporciona funciones helper para construir estructuras

2. gecode__bridge.cpp (Lado C++)

Proposito: Wrapper C++ que expone una API C para ser llamada desde Pascal.

Ubicacion: /home/rodo/gecode/src/gecode__bridge.cpp

Caracteristicas: - Usa `extern "C" { ... }` para exportar simbolos con C linkage - Define clase `CSPModel` que hereda de `Gecode::Space` - Implementa funciones C que crean/manipulan objetos C++ - Usa `#pragma GCC visibility push(hidden)` para ocultar simbolos internos

Flujo de Datos

Pascal Program

?

Llama: `csp_create(vars, n)`

?

`gecode__bridge.cpp: extern "C" void* csp_create(...)`

?

Crea: `new CSPModel(vars, n)`

?

`Gecode: IntVar, IntVarArray, branch()`

?

Retorna: `void*` (puntero opaco)

?

Pascal: `Model := Pointer` (maneja como handle opaco)

Estructuras de Datos Compartidas

Las estructuras deben ser **binarias compatibles** entre Pascal y C++. Esto se logra con:

- Pascal: {\$PACKRECORDS C} – alineación C
- C++: Estructuras POD (Plain Old Data) sin virtual tables

TCSPVar / Variable

Pascal (UGecodeBridge.pas):

```
type
  TCSPVar = record
    Name      : array[0..63] of Char;
    MinDomain : LongInt;
    MaxDomain : LongInt;
  end;
  PCSPVar = ^TCSPVar;
```

C++ (gecode_bridge.cpp):

```
struct Variable {
    char name[64];
    int  min_domain;
    int  max_domain;
};
```

Uso:

```
var
  V: TCSPVar;
begin
  V := CSPMakeVar('x', 1, 10); // Helper Pascal
  Model := csp_create(@V, 1);  // Llamada a C++
end;
```

TCSPConstraint / Constraint

Pascal (UGecodeBridge.pas):

```
type
  TCSPConstraint = record
    CType      : LongInt;           { CT_* }
    Var1       : array[0..63] of Char;
    Var2       : array[0..63] of Char;
    Constant   : LongInt;
    Lo, Hi      : LongInt;
    LoOpen, HiOpen : ByteBool;
    SetVals    : array[0..99] of LongInt;
    SetSize    : LongInt;
    LinVars    : array[0..19, 0..63] of Char;
    LinCoefs   : array[0..19] of LongInt;
```

```

LinNVars : LongInt;
LinRHS   : LongInt;
ADiffVars : array[0..49, 0..63] of Char;
ADiffNVars : LongInt;
end;

```

C++ (gecode_bridge.cpp):

```

struct Constraint {
    int  type;           // ConstraintType
    char var1[64];
    char var2[64];
    int  constant;
    int  lo, hi;
    bool lo_open, hi_open;
    int  set_vals[100];
    int  set_size;
    char lin_vars[20][64];
    int  lin_coefs[20];
    int  lin_nvars;
    int  lin_rhs;
    char adiff_vars[50][64];
    int  adiff_nvars;
};

```

Nota: Solo se llenan los campos relevantes al tipo de restriccion (discriminador CType/type).

TCSPSolution / Solution

Pascal:

```

type
  TCSPSolution = record
    Names   : array[0..49, 0..63] of Char;
    Values  : array[0..49] of LongInt;
    NumVars : LongInt;
  end;

```

C++:

```

struct Solution {
    char names[50][64];
    int  values[50];
    int  num_vars;
};

```

API Exportada desde C++

La API C exportada es **minimalista y eficiente**, exponiendo solo lo necesario:

Funciones Principales

csp_create

```
void* csp_create(Variable* vars, int n);
```

Proposito: Crear un modelo CSP con N variables.

Parametros: - vars: Array de definiciones de variables - n: Numero de variables

Retorno: Puntero opaco al modelo (handle)

Uso Pascal:

```
var
  Vars: array[0..1] of TCSPVar;
  Model: Pointer;
begin
  Vars[0] := CSPMakeVar('x', 1, 5);
  Vars[1] := CSPMakeVar('y', 0, 10);
  Model := csp_create(@Vars[0], 2);
end;
```

csp_add_constraint

```
int csp_add_constraint(void* model, Constraint* constraint);
```

Proposito: Agregar una restriccion al modelo.

Parametros: - model: Handle del modelo - constraint: Puntero a estructura de restriccion

Retorno: 1 si OK, 0 si error

Uso Pascal:

```
var
  C: TCSPConstraint;
begin
  C := CSPEq('x', 5); // x = 5
  if csp_add_constraint(Model, @C) = 0 then
    WriteLn('Error al agregar restriccion');
end;
```

csp_solve_first

```
int csp_solve_first(void* model, Solution* sol);
```

Proposito: Encontrar la primera solucion.

Retorno: 1 si encontro solucion, 0 si no hay

csp_solve_all

```
int csp_solve_all(void* model, Solution* solutions, int max_sols);
```

Proposito: Encontrar todas las soluciones (hasta max_sols).

Retorno: Numero de soluciones encontradas

Uso Pascal:

```
var
  Solutions: array[0..99] of TCSPSolution;
  NumSols: Integer;
  i, j: Integer;
begin
  NumSols := csp_solve_all(Model, @Solutions[0], 100);

  for i := 0 to NumSols - 1 do
  begin
    for j := 0 to Solutions[i].NumVars - 1 do
      Write(Format('%s=%d ', [
        PChar(@Solutions[i].Names[j]),
        Solutions[i].Values[j]
      ]));
    WriteLn;
  end;
end;
```

csp_count_solutions

```
int csp_count_solutions(void* model);
```

Proposito: Contar soluciones sin devolverlas (mas eficiente).

Retorno: Numero total de soluciones

csp_count_with_constraint

```
int csp_count_with_constraint(void* model, Constraint* constraint);
```

Proposito: Contar soluciones con una restriccion adicional **sin modificar** el modelo.

Uso: Exploracion del espacio de soluciones (usado por GecodeInfo).

csp_debug_domains

```
int csp_debug_domains(void* model);
```

Proposito: Propagar restricciones e imprimir dominios por stderr (diagnostico).

Retorno: 0 si FAILED, 1 si OK

csp_free

```
void csp_free(void* model);
```

Proposito: Liberar memoria del modelo.

Tipos de Restricciones Soportadas

El bridge soporta **21 tipos de restricciones** que cubren la mayoría de casos de uso CSP:

Restricciones Basicas

```
const
  CT_EQ    = 0;    // var1 = var2 | const
  CT_NEQ   = 1;    // var1 <> var2 | const
  CT_LT    = 2;    // var1 < var2 | const
  CT_GT    = 3;    // var1 > var2 | const
  CT_LE    = 4;    // var1 <= var2 | const
  CT_GE    = 5;    // var1 >= var2 | const
```

Ejemplo:

```
C := CSPEq('x', 10);      // x = 10
C := CSPEqVar('x', 'y');  // x = y
C := CSPLt('x', 100);     // x < 100
```

Restricciones de Dominio

```
const
  CT_IN_INTERVAL = 6;    // lo <= var <= hi
  CT_IN_SET      = 7;    // var IN {v0, v1, ...}
```

Ejemplo:

```
C := CSPInterval('x', 1, 10);      // x ? [1, 10]
C := CSPInterval('y', 0, 5, True, False); // y ? (0, 5]
C := CSPInSet('color', [1, 3, 5, 7]); // color ? {1,3,5,7}
```

Restricciones Lineales

```
const
  CT_LINEAR_EQ  = 8;    // sum(coef[i]*var[i]) = rhs
  CT_LINEAR_LE  = 9;    // sum(coef[i]*var[i]) <= rhs
  CT_LINEAR_GE  = 10;   // sum(coef[i]*var[i]) >= rhs
  CT_LINEAR_LT  = 11;   // sum(coef[i]*var[i]) < rhs
  CT_LINEAR_GT  = 12;   // sum(coef[i]*var[i]) > rhs
  CT_LINEAR_NEQ = 13;   // sum(coef[i]*var[i]) <> rhs
```

Ejemplo:

```
// x + y = 10
C := CSPAddEq('x', 'y', 10);

// 2*x - 3*y = 5
C := CSPLinEq2('x', 2, 'y', -3, 5);
```

```
// General:  $3x + 5y - 2z \leq 100$ 
C := CSPLinear(CT_LINEAR_LE,
               ['x', 'y', 'z'],
               [3, 5, -2],
               100);
```

Restricciones de Valor Absoluto

```
const
CT_ABS_EQ = 14;    //  $\text{abs}(\text{var}) = \text{const}$ 
CT_ABS_LE = 15;    //  $\text{abs}(\text{var}) \leq \text{const}$ 
CT_ABS_GE = 16;    //  $\text{abs}(\text{var}) \geq \text{const}$ 
```

Ejemplo:

```
C := CSPAbsEq('x', 10);    //  $|x| = 10 \rightarrow x \in \{-10, 10\}$ 
C := CSPAbsLe('y', 5);    //  $|y| \leq 5 \rightarrow y \in [-5, 5]$ 
```

Restricciones de Distancia

```
const
CT_DIST_EQ = 17;    //  $|\text{var1} - \text{var2}| = \text{const}$ 
CT_DIST_LE = 18;    //  $|\text{var1} - \text{var2}| \leq \text{const}$ 
CT_DIST_GE = 19;    //  $|\text{var1} - \text{var2}| \geq \text{const}$ 
```

Ejemplo:

```
C := CSPDistEq('x', 'y', 5);    //  $|x - y| = 5$ 
C := CSPDistLe('a', 'b', 10);   //  $|a - b| \leq 10$ 
```

Restriccion Global

```
const
CT_ALL_DIFF = 20;    //  $\text{all\_different}([\text{vars}])$ 
```

Ejemplo:

```
C := CSPAllDiff(['x', 'y', 'z', 'w']);
// Todas las variables deben tener valores distintos
```

Compilacion Monolitica

El sistema soporta **compilacion monolitica** para crear ejecutables standalone sin dependencias externas.

Modos de Compilacion

Modo Estatico

Requisitos: - Gecode compilado como biblioteca estatica (.a) - Variable `GECODE_HOME` apuntando a instalacion estatica

Resultado: - Ejecutable de ~8-12 MB - **Sin dependencias** de `libgecode*.so` - Solo depende de `libc/libm` del sistema - Completamente portable

Modo Dinamico

Requisitos: - Gecode instalado del sistema (`/usr/lib/libgecode*.so`)

Resultado: - Ejecutable de ~500 KB - Requiere `libgecode*.so` en runtime - Mas pequeno pero menos portable

Script `build_monolithic.sh`

Ubicacion: `/home/rodo/gecode/scripts/build_monolithic.sh`

Uso:

```
# Compilar programa especifico
./scripts/build_monolithic.sh src/TestGecodeBridge.pas

# Compilar con GECODE_HOME personalizado
GECODE_HOME=$HOME/gecode-static ./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

Proceso de Compilacion

El script realiza **3 pasos**:

Paso 1: Compilar Bridge C++

```
g++ -c src/gecode_bridge.cpp \
    -o obj/gecode_bridge.o \
    -I$GECODE_HOME/include \
    -std=c++17 \
    -O2 \
    -DNDEBUG \
    -fvisibility=hidden \
    -ffunction-sections \
    -fdata-sections \
    -fno-stack-protector
```

Flags importantes: - `-fvisibility=hidden`: Oculta simbolos internos (solo exporta `csp_*`) - `-ffunction-sections`: Permite garbage collection de secciones no usadas - `-fdata-sections`: Igual para datos

Paso 2: Compilar Pascal

```
fpc TestGecodeBridge.pas -O2 -Cn -FU obj/ -Fu obj/ -Fu src/
```

Genera: - `link*.res`: Script de linkeo con lista de objetos - `obj/*.o`: Objetos Pascal compilados - `obj/*.ppu`: Units compiladas

Post-procesamiento:

```
# Ajustar paths en link*.res
sed -i "s|^([A-Za-z][A-Za-z0-9_]*\.o\)$|obj/\1|g" link*.res
```

Paso 3: Linkeo Final

Modo estatico:

```
/usr/bin/ld.bfd -b elf64-x86-64 -m elf_x86_64 -s \
-o bin/TestGecodeBridge \
-T link*.res -e _start \
--dynamic-linker /lib64/ld-linux-x86-64.so.2 \
--gc-sections \
-L$GECODE_HOME/lib \
-Bstatic \
-lgecodeminimodel \
-lgecodeint \
-lgecodesearch \
-lgecodekernel \
-lgecodesupport \
-Bdynamic \
-lstdc++ -lgcc_s -lc
```

Flags importantes: - `-Bstatic`: Linkeo estatico para Gecode - `-Bdynamic`: Linkeo dinamico para `libstdc++/libc` - `--gc-sections`: Eliminar secciones no usadas - `-s`: Strip simbolos debug

Verificacion

```
# Ver tamano
ls -lh bin/TestGecodeBridge
# ~8-12 MB en modo estatico

# Ver dependencias
ldd bin/TestGecodeBridge
# Solo debe mostrar:
#   linux-vdso.so.1
#   libc.so.6
#   libm.so.6
```

```
# libstdc++.so.6 (si no se usa -static-libstdc++)
# /lib64/ld-linux-x86-64.so.2

# Ver simbolos exportados del bridge
nm -D bin/TestGecodeBridge | grep " T csp_"
# Debe listar: csp_create, csp_add_constraint, etc.
```

Programas de Prueba

El proyecto incluye **4 programas de prueba** que demuestran el uso del bridge:

1. TestGecodeBridge

Proposito: Resolver CSP completo desde JSON usando el motor Gecode.

Ubicacion: /home/rodo/gecode/src/TestGecodeBridge.pas

Uso:

```
# Resolver archivo especifico
./bin/TestGecodeBridge grafo.json

# Correr todos los tests
./bin/TestGecodeBridge
```

Funcionalidad: - Lee grafo JSON (salida de JsonToGraph) - Traduce variables y restricciones a modelo Gecode - Ejecuta busqueda DFS - Analiza espacio de soluciones - Retorna JSON con analisis

Salida:

```
{
  "status": "ok",
  "total_solutions": 252,
  "analysis": [
    {
      "name": "ZONA",
      "type": "set",
      "if_fixed": [
        { "value": "NORTE", "solutions": 126 },
        { "value": "SUR", "solutions": 126 }
      ]
    }
  ]
}
```

2. GecodeInfo

Proposito: Analizar el espacio de soluciones sin enumerar todas.

Ubicacion: /home/rodo/gecode/src/GecodeInfo.pas

Uso:

```
./bin/GecodeInfo grafo.json
```

Funcionalidad: - Para cada variable no-determinada - Calcula cuantas soluciones existen si se fija a cada valor posible - Genera mapa de distribucion del espacio de busqueda

Salida:

```
{
  "status": "ok",
  "total_solutions": 100,
  "analysis": [
    {
      "name": "X",
      "type": "integer",
      "if_fixed": [
        { "value": 1, "solutions": 10 },
        { "value": 2, "solutions": 20 },
        { "value": 3, "solutions": 30 },
        { "value": 4, "solutions": 25 },
        { "value": 5, "solutions": 15 }
      ]
    }
  ]
}
```

Diferencia con TestGecodeBridge: - GecodeInfo: **analisis** del espacio - TestGecodeBridge: **enumeracion** + analisis

3. TestComplejo

Proposito: Demostracion de dos problemas CSP clasicos usando el bridge directamente desde Pascal (sin JSON).

Ubicacion: /home/rodo/gecode/src/TestComplejo.pas

Uso:

```
./bin/TestComplejo
```

Problema 1: SEND + MORE = MONEY

Descripcion: Criptoaritmetica clasica.

Restricciones:

```
// 8 variables: S, E, N, D, M, O, R, Y
// Dominios: [0..9] excepto S, M ? [1..9]

// All different
CSPAllDiff(['S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y'])

// Ecuacion: SEND + MORE = MONEY
// 1000*S + 100*E + 10*N + D +
// 1000*M + 100*O + 10*R + E =
// 10000*M + 1000*O + 100*N + 10*E + Y

// Simplificado:
CSPLinear(CT_LINEAR_EQ,
```



```
['S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y'],
[1000, 91, -90, 1, -9000, -900, 10, -1],
0)
```

Solucion unica:

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

Problema 2: Cuadrado Magico 3x3

Descripcion: Colocar numeros 1..9 en cuadrícula 3x3 donde todas las filas, columnas y diagonales sumen 15.

Restricciones:

```
// 9 variables: C11, C12, C13, C21, C22, C23, C31, C32, C33
// Todas en [1..9] y distintas
CSPAllDiff([...])

// Filas suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C12', 'C13'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C21', 'C22', 'C23'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C31', 'C32', 'C33'], [1,1,1], 15)

// Columnas suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C21', 'C31'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C12', 'C22', 'C32'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C13', 'C23', 'C33'], [1,1,1], 15)

// Diagonales suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C22', 'C33'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C13', 'C22', 'C31'], [1,1,1], 15)
```

Soluciones: 8 (rotaciones y reflexiones del cuadrado base)

4. TestIntervalos

Proposito: Resolver sistema de restricciones lineales con dominios continuos discretizados.

Ubicacion: /home/rodo/gecode/src/TestIntervalos.pas

Uso:

```
./bin/TestIntervalos
```

Sistema:

Variables:

```
x ? [-2, 5] (discretizado en paso 0.1)
y ? [-1, 6] (discretizado en paso 0.1)
```

Restricciones:

$$5x + 3y \leq [1, 3]$$

$$6x + 10y \leq [10, 50]$$

Discretizacion:

```
// Multiplicar por 10 para evitar punto flotante
```

```
VarX := CSPMakeVar('x', -20, 50);    // x x 10
```

```
VarY := CSPMakeVar('y', -10, 60);    // y x 10
```

```
// Restriccion: 5x + 3y ≤ [1, 3]
```

```
// -> (5x(x/10) + 3x(y/10)) x 10 ≤ [10, 30]
```

```
// -> 5x + 3y ≤ [10, 30]
```

Salida: - Estadísticas de region factible - Plot ASCII de la region - Lista de puntos solucion

Ejemplos de Uso

Ejemplo 1: Resolver CSP Simple

```
program EjemploSimple;

uses UGecodeBridge;

var
  Model: Pointer;
  Vars: array[0..1] of TCSPVar;
  C: TCSPConstraint;
  Sol: TCSPSolution;
  i: Integer;

begin
  // Definir variables
  Vars[0] := CSPMakeVar('x', 1, 5);
  Vars[1] := CSPMakeVar('y', 1, 10);

  // Crear modelo
  Model := csp_create(@Vars[0], 2);

  // Restriccion:  $x + y = 10$ 
  C := CSPAddEq('x', 'y', 10);
  csp_add_constraint(Model, @C);

  // Resolver (primera solucion)
  if csp_solve_first(Model, @Sol) = 1 then
  begin
    WriteLn('Solucion:');
    for i := 0 to Sol.NumVars - 1 do
      WriteLn('  ', PChar(@Sol.Names[i]), ' = ', Sol.Values[i]);
    end
  else
    WriteLn('Sin solucion');

  // Liberar
  csp_free(Model);
end.
```

Ejemplo 2: Todas las Soluciones

```
program TodasSoluciones;

uses UGecodeBridge;

var
```

```

Model: Pointer;
Vars: array[0..2] of TCSPVar;
C: TCSPConstraint;
Solutions: array[0..99] of TCSPSolution;
NumSols, i, j: Integer;

begin
    // Variables: x, y, z ? [1, 3]
    Vars[0] := CSPMakeVar('x', 1, 3);
    Vars[1] := CSPMakeVar('y', 1, 3);
    Vars[2] := CSPMakeVar('z', 1, 3);

    Model := csp_create(@Vars[0], 3);

    // Todas diferentes
    C := CSPAllDiff(['x', 'y', 'z']);
    csp_add_constraint(Model, @C);

    // x + y + z = 6
    C := CSPLinear(CT_LINEAR_EQ,
                  ['x', 'y', 'z'],
                  [1, 1, 1],
                  6);
    csp_add_constraint(Model, @C);

    // Resolver todas
    NumSols := csp_solve_all(Model, @Solutions[0], 100);

    WriteLn('Soluciones: ', NumSols);
    for i := 0 to NumSols - 1 do
    begin
        Write(Format('%2d. ', [i + 1]));
        for j := 0 to Solutions[i].NumVars - 1 do
            Write(Format('%s=%d ', [
                PChar(@Solutions[i].Names[j]),
                Solutions[i].Values[j]
            ]));
        WriteLn;
    end;

    csp_free(Model);
end.

```

Ejemplo 3: Exploracion del Espacio

```

program ExploracionEspacio;

```

```

uses UGecodeBridge;

var
  Model: Pointer;
  Vars: array[0..1] of TCSPVar;
  C: TCSPConstraint;
  Total, Count, Val: Integer;

begin
  // x, y ? [1, 10]
  Vars[0] := CSPMakeVar('x', 1, 10);
  Vars[1] := CSPMakeVar('y', 1, 10);

  Model := csp_create(@Vars[0], 2);

  // x + y <= 15
  C := CSPAddLe('x', 'y', 15);
  csp_add_constraint(Model, @C);

  // Total de soluciones
  Total := csp_count_solutions(Model);
  WriteLn('Total: ', Total, ' soluciones');
  WriteLn;

  // ?Cuantas soluciones si x = 1? x = 2? etc.
  WriteLn('Análisis:');
  for Val := 1 to 10 do
    begin
      C := CSPEq('x', Val);
      Count := csp_count_with_constraint(Model, @C);
      WriteLn(' Si x = ', Val, ' -> ', Count, ' soluciones');
    end;

  csp_free(Model);
end.

```

Resumen de la Integracion

Ventajas de esta Arquitectura

1. **Separacion de responsabilidades**
 - Pascal: Logica de negocio, I/O, pipeline
 - C++: Solo interfaz a Gecode
 - C: Funciones matematicas low-level
2. **Ejecutables monoliticos**
 - Un solo archivo standalone
 - Sin DLLs externas
 - Portable entre sistemas
3. **API minimalista**
 - Solo 8 funciones exportadas
 - Estructuras simples (POD)
 - Sin overhead de runtime
4. **Eficiencia**
 - Garbage collection de simbolos no usados
 - Linkeo estatico de Gecode
 - Optimizaciones de compilador
5. **Mantenibilidad**
 - Bridge C++ aislado en un solo archivo
 - Interface Pascal clara y tipada
 - Helpers Pascal para construccion facil

Limitaciones

1. **Variables solo enteras**
 - No soporta float directamente (se discretiza)
 - No soporta set variables de Gecode (se mapean a int)
2. **Estructuras de tamano fijo**
 - Arrays con limites predefinidos
 - No dinamicos para compatibilidad C
3. **Linkeo manual**
 - Requiere script de compilacion especifico
 - No usa build system estandar

Flujo Completo

1. Escribir programa Pascal
 - Usar UGecodeBridge
 - Crear variables con CSPMakeVar
 - Crear restricciones con CSP* helpers
 - Llamar csp_create, csp_add_constraint, csp_solve_*
2. Compilar con build_monolithic.sh
 - Compila gecode_bridge.cpp -> .o
 - Compila Pascal -> .o
 - Linkea todo + Gecode estatico

3. Ejecutar

- Ejecutable standalone
- Sin dependencias
- Totalmente portable

Conclusion: La integracion Pascal/C++/Gecode permite combinar la **productividad de Pascal** para el pipeline con el **poder de Gecode** para resolucion CSP, generando **ejecutables eficientes y portables**.